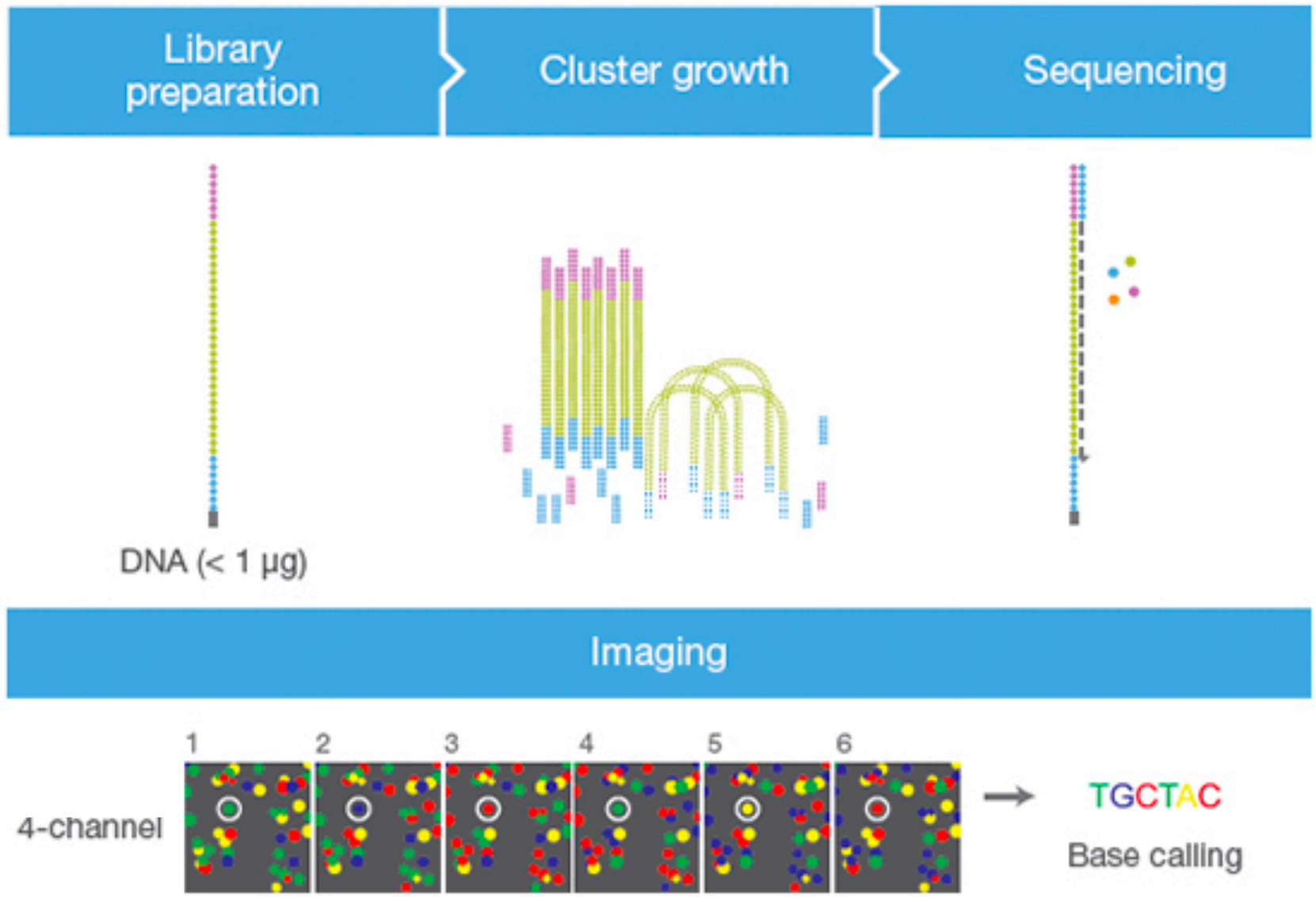
Assembly project / Biopython

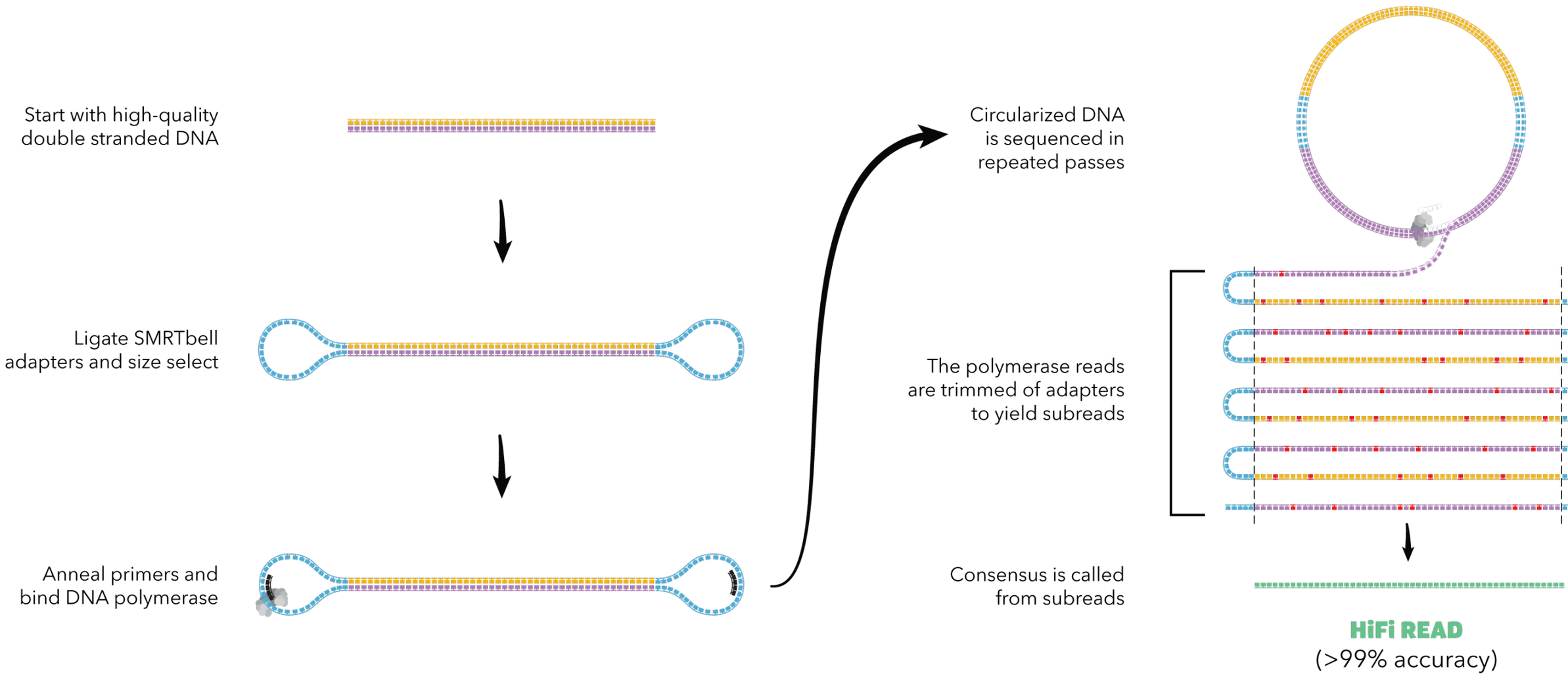
- Brief introduction to genome assambly.
- Walking through the genome assembly project.
- Working with seuquences using Biopython

Illumina
(next-gen sequencing)



Reads of ~ 200 bp

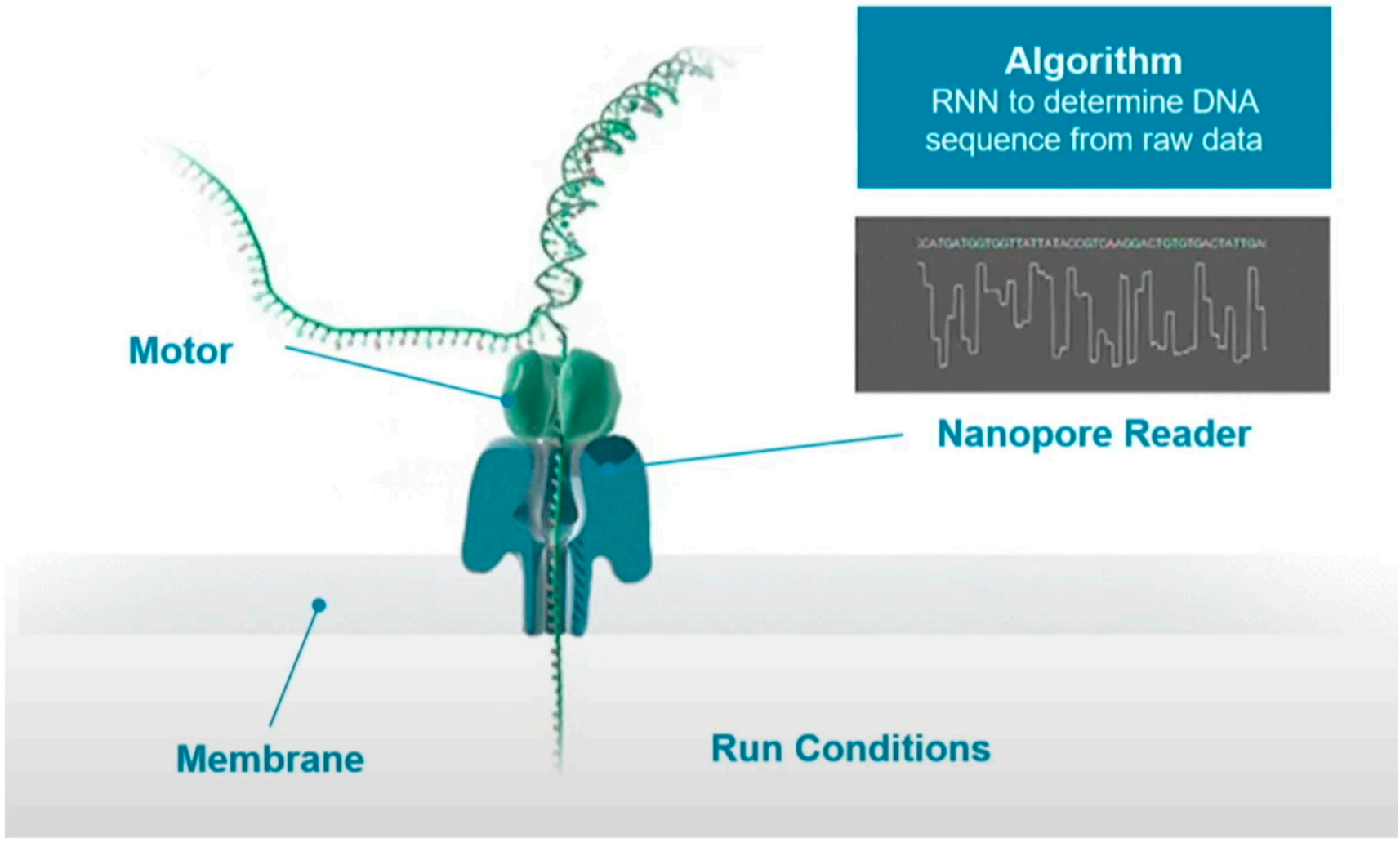
PacBio HiFi



Reads of ~ 30,000 bp

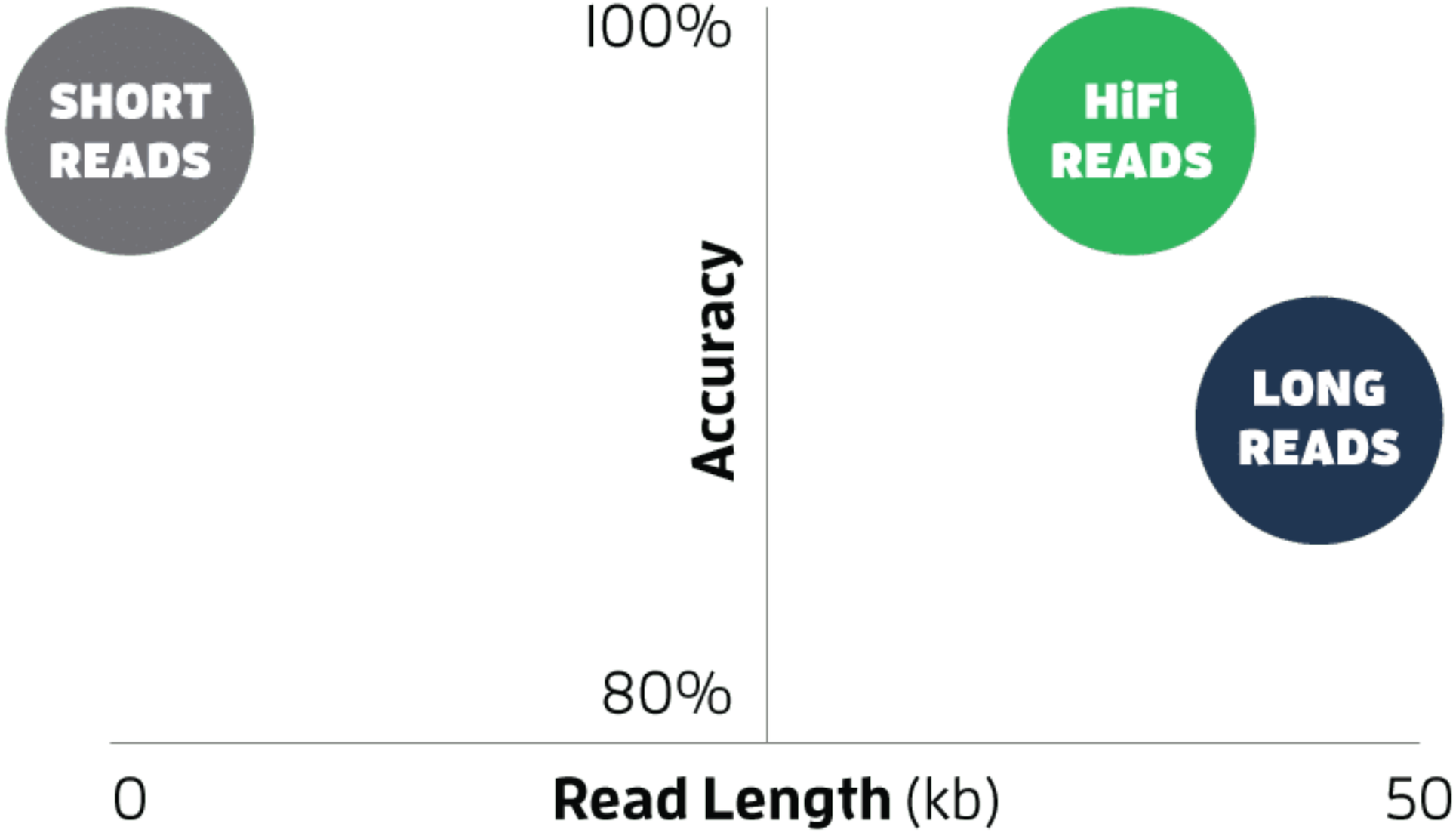
YouTube movie

Oxford nanopore



Reads of ~ 200,000 bp

YouTube movie


SHORT READS
100%
Accuracy
HiFi READS
LONG READS
80%
0
Read Length (kb)
50

10 years
$3,000,000,000



Days
$1,000

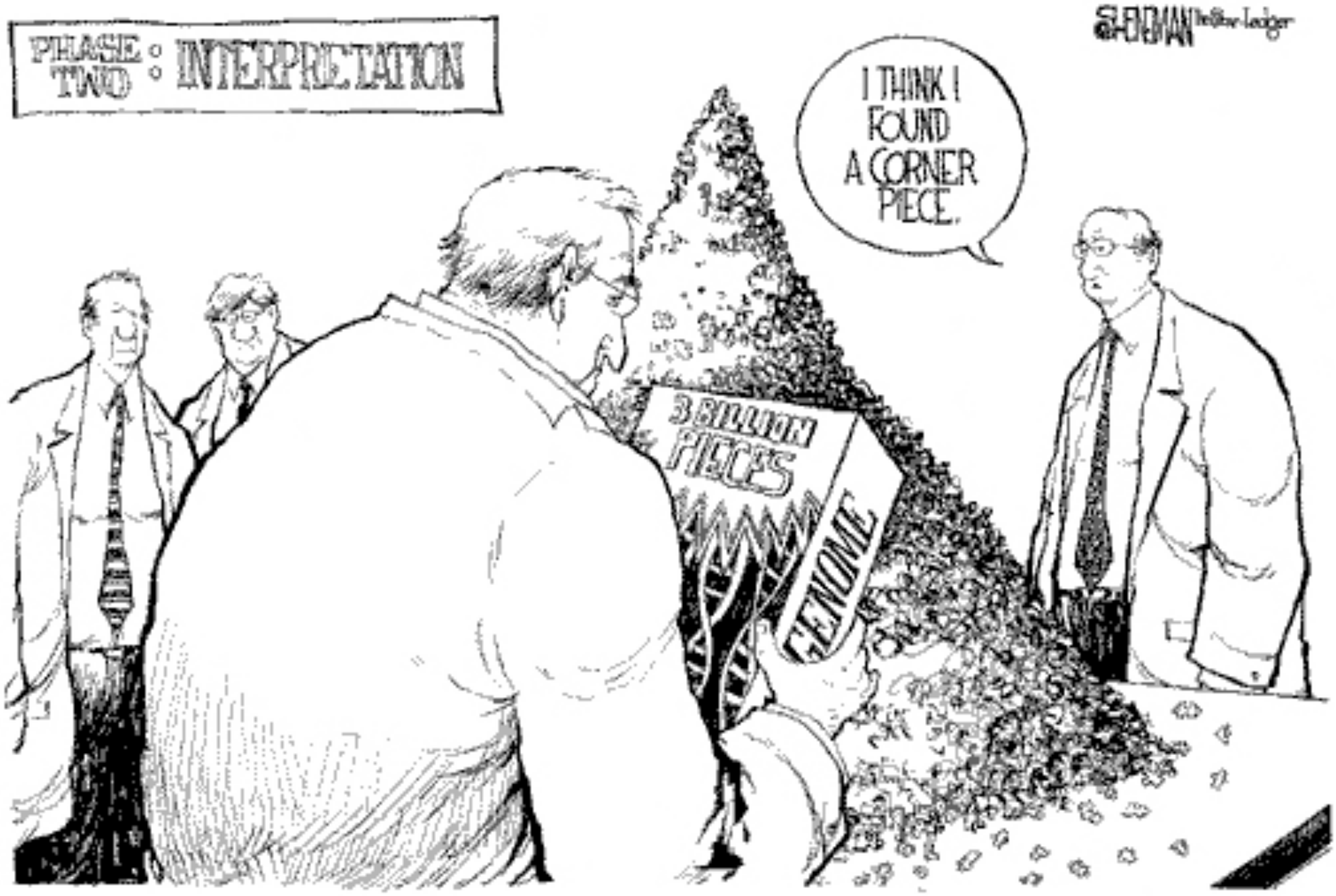


Pac-Bio HiFi

2001

2026

Assembly vs resequencing

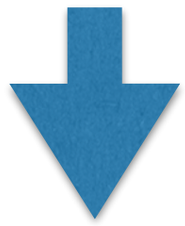


Project:

Genome assembly

Genome assembly

```
Read1:      AGGTCGTAG
Read2:           CGTAGAGCTGGGAG
Read3:                     GGGAGGTTGAAA
```



```
genome:     AGGTCGTAGAGCTGGGAGGTTGAAA
```

Sequencing reads

```
Read1 GGCTCCCCACGGGGTACCCATAACTTGACAGTAGATCTCGTCCAGACCCCTAGC
Read2 CTTTACCCGGAAGAGCGGGACGCTGCCCTGCGCGATTCCAGGCTCCCCACGGG
Read4 TGCGAGGGAAGTGAAGTATTTGACCCTTTACCCGGAAGAGCG
Read3 GTCTTCAGTAGAAAATTGTTTTTTTCTTCCAAGAGGTCGGAGTCGTGAACACATCAGT
Read5 CGATTCCAGGCTCCCCACGGGGTACCCATAACTTGACAGTAGATCTC
Read6 TGACAGTAGATCTCGTCCAGACCCCTAGCTGGTACGTCTTCAGTAGAAAATTGTTTTTTTCTTCCAAGAGGTCGGAGT
```

1. No sequencing errors

AGGTCGTAG

CGTAGAGCTGGGAG

GGGAGGTTGAAA

Kun perfekte overlap

2. No nested reads

```
xxxxxxxxxxx
     xxxxxxxxxxxx
xxxxxxxxxxxx
```





```
xxxxxxxxxxxxxxx
    xxxxxxxxx
```



Ingen reads inde i andre reads

3. Overlaps of 3 bases do not occur at random

xxxxxxxxxx
xxxxxxxxxx
2

Muligvis tilfældige overlap

xxxxxxxxxxxx
xxxxxxxxxxxx
3

Sandt overlap

Vi leder efter overlap på mindst tre baser

4. No holes in our assembly

Reads

```
AGGTCGG
     CGTAGGAG
          GAGGTTA     GGTCGG
                      CGGAGT
```

Genom **AGGTCGGAGGAGGTTA**TAGT**GGTCGGAGT**

Hul der ikke er dækket af sammenhængende reads

Do a quick decomposition

Part 1

`read_data`

Part 4

`assemble_genome`

`reconstruct_sequence`

Part 2

`get_all_overlaps`

`get_overlap`

Part 3

`find_first_read`

`get_left_overlaps`

`find_order_of_reads`

`find_key_for_largest_value`

Read in data

Write a function, *read_data*, that takes one argument:

1. A string, which is the name of the data file.

The function must return:

- A dictionary, where the keys are the names of reads and the values are the associated read sequences. Both keys and values must be strings.

```
{'Read1': 'GGCTCCCCACGGGGTACCCATAACTTGACAGTAGATCTCGTCCAGACCCCTAGC',
 'Read3': 'GTCTTCAGTAGAAAATTGTTTTTTTCTTCCAAGAGGTCGGAGTCGTGAACACATCAGT',
 'Read2': 'CTTTACCCGGAAGAGCGGGACGCTGCCCTGCGCGATTCCAGGCTCCCCACGGG',
 'Read5': 'CGATTCCAGGCTCCCCACGGGGTACCCATAACTTGACAGTAGATCTC',
 'Read4': 'TGCGAGGGAAGTGAAGTATTTGACCCTTTACCCGGAAGAGCG',
 'Read6': 'TGACAGTAGATCTCGTCCAGACCCCTAGCTGGTACGTCTTCAGTAGAAAATTGTTTTTTTCTTCCAAGAGGTCGGAGT'}
```

Read in data

```
def read_data(file_name):
    input_file = open(file_name)
    ...
    for line in input_file:
        ...
    ...

    input_file.close()
    return ...
```

```
Read1 GGCTCCCCACGGGGTACCCATAACTTGACAGTAGATCTCGTCCAGACCCCTAGC
Read2 CTTTACCCGGAAGAGCGGGACGCTGCCCTGCGCGATTCCAGGCTCCCCACGGG
Read4 TGCGAGGGAAGTGAAGTATTTGACCCTTTACCCGGAAGAGCG
Read3 GTCTTCAGTAGAAAATTGTTTTTTTCTTCCAAGAGGTCGGAGTCGTGAACACATCAGT
Read5 CGATTCCAGGCTCCCCACGGGGTACCCATAACTTGACAGTAGATCTC
Read6 TGACAGTAGATCTCGTCCAGACCCCTAGCTGGTACGTCTTCAGTAGAAAATTGTTTTTTTCTTCCAAGAGGTCGGAGT
```

Find the largest overlap



AGGTCGTAG

CGTAGAGCTGGGAG

Find the largest overlap

```
Read1:      AGGTCGTAG
Read2:  CGTAGAGCTGGGAG
```

Find the largest overlap

```
Read1:      AGGTCGTAG
Read2:   CGTAGAGCTGGGAG
```

Find the largest overlap

```
Read1:      AGGTCGTAG
Read2:   CGTAGAGCTGGGAG
```

Find the largest overlap

```
Read1:     AGGTCGTAG
Read2:          CGTAGAGCTGGGAG
```

Find the largest overlap

```
Read1:    AGGTCGTAG
Read2:          CGTAGAGCTGGGAG
```

Find the largest overlap

```
Read1:    AGGTCGTAG
Read2:        CGTAGAGCTGGGAG
```

Exercise

```
Read1:    AGGTCGTAG
Read2:        CGTAGAGCTGGGAG
```

Brug slicing til at få de sidste 5 karakter af en sekvens og de første 5 karakterer.

Exercise

```
Read1:    AGGTCGTAG
Read2:        CGTAGAGCTGGGAG
```

Skriv en funktion der finder det største overlap mellem en vilkårlig venstre og højre sekvens.

```
get_overlap('AGGTCGTAG', 'CGTAGAGCTGGGAG')
```

skal returnere: `'CGTAG'`

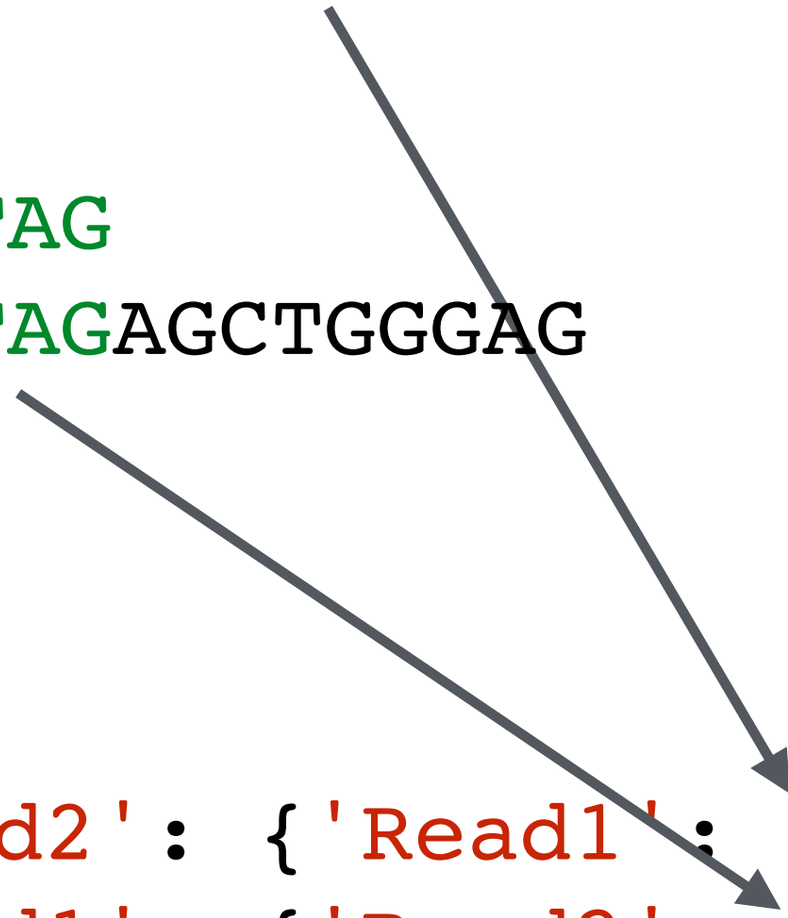
Det modsatte overlap

```
1:                AGGTCGTAG
2:  CGTAGAGCTGGGAG
```

Kan vi bruge vores funktion til at finde det overlap?

```
Read2:  CGTAGAGCTGGGAG
Read1:                AGGTCGTAG


Read1:  AGGTCGTAG
Read2:      CGTAGAGCTGGGAG
```



```
overlaps = {'Read2': {'Read1': 2},
            'Read1': {'Read2': 5}}

print(overlaps['Read1']['Read2'])
```

Alle overlap

```
{'Read1': {'Read3':  0, 'Read2':  1, 'Read5':  1, 'Read4': 0, 'Read6': 29},
 'Read3': {'Read1':  0, 'Read2':  0, 'Read5':  0, 'Read4': 1, 'Read6':  1},
'Read2': {'Read1': 13, 'Read3':  1, 'Read5': 21, 'Read4': 0, 'Read6':  0},
'Read5': {'Read1': 39, ‘Read3':  0, 'Read2':  1, 'Read4': 0, 'Read6': 14},
'Read4': {'Read1':  1, 'Read3':  1, 'Read2': 17, 'Read5': 2, 'Read6':  0},
'Read6': {'Read1':  0, 'Read3': 43, 'Read2':  0, 'Read5': 0, 'Read4':  1}}
```

`get_all_overlaps`

Write a function, **`get_all_overlaps`**, that takes one argument:

1. A dictionary with read data as returned by **`read_data`**.

The function must return:

- A dictionary of dictionaries, specifying the number of overlapping bases for a pair of reads in a specific left-right orientation. Computing the overlap of a read to itself is meaningless and must not be included.

```
{'Read1': {'Read3':  0, 'Read2':  1, 'Read5':  1, 'Read4': 0, 'Read6': 29},
 'Read3': {'Read1':  0, 'Read2':  0, 'Read5':  0, 'Read4': 1, 'Read6':  1},
 'Read2': {'Read1': 13, 'Read3':  1, 'Read5': 21, 'Read4': 0, 'Read6':  0},
 'Read5': {'Read1': 39, ‘Read3':  0, 'Read2':  1, 'Read4': 0, 'Read6': 14},
 'Read4': {'Read1':  1, 'Read3':  1, 'Read2': 17, 'Read5': 2, 'Read6':  0},
 'Read6': {'Read1':  0, 'Read3': 43, 'Read2':  0, 'Read5': 0, 'Read4':  1}}
```

Print det pænt

Right position:

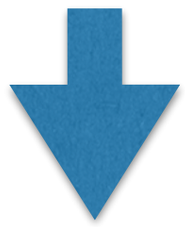
Left position

	Read1	Read2	Read3	Read4	Read5	Read6
Read1	-	1	0	0	1	29
Read2	13	-	1	0	21	0
Read3	0	0	-	1	0	1
Read4	1	17	1	-	2	0
Read5	39	1	0	0	-	14
Read6	0	0	43	1	0	-

Hvordan finder vi den rigtige rækkefølge af reads?

The right order of reads

```
Read1:      AGGTCGTAG
Read2:           CGTAGAGCTGGGAG
Read3:                     GGGAGGTTGAAA
```



```
genome:     AGGTCGTAGAGCTGGGAGGTTGAAA
```

Find the first read

```
AGGTCGTAG
      XXXXXXXXXXXXX
```

The first (leftmost) read has a good match to the right…

```
XXXXXXXX
       AGGTCGTAG
```

… but only short random matches to the left

Find the first read

Right position:

Left position

	Read1	Read2	Read3	Read4	Read5	Read6
Read1	-	1	0	0	1	29
Read2	13	-	1	0	21	0
Read3	0	0	-	1	0	1
Read4	1	17	1	-	2	0
Read5	39	1	0	0	-	14
Read6	0	0	43	1	0	-

Find the first read

```
Read1:      AGGTCGTAG
Read2:          CGTAGAGCTGGGAG
Read3:                      GGGAGGTTGAAA
```

Overlaps of read1 when read1 is in the *right* position:

```
overlaps = {'Read1': {'Read2': 5, 'Read3':  1},
            'Read2': {'Read1': 2, 'Read3':  5},
            'Read3': {'Read1': 1, 'Read2':  0}}
```

Get left overlaps

Write a function, **`get_left_overlaps`**, that takes two arguments:

1. A dictionary of dictionaries as returned from **`get_all_overlaps`**.
2. A string, which represents the name of a read.

The function must return:

- A sorted list of integers, which represent the overlaps of other reads to its left end.

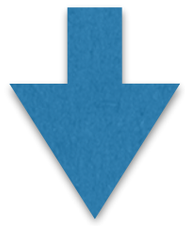
Find the next read

```
{'Read1': {'Read3':  0, 'Read2':  1, 'Read5':  1, 'Read4': 0, 'Read6': 29},
 'Read3': {'Read1':  0, 'Read2':  0, 'Read5':  0, 'Read4': 1, 'Read6':  1},
'Read2': {'Read1': 13, 'Read3':  1, 'Read5': 21, 'Read4': 0, 'Read6':  0},
'Read5': {'Read1': 39, 'Read3':  0, 'Read2':  1, 'Read4': 0, 'Read6': 14},
'Read4': {'Read1':  1, 'Read3':  1, 'Read2': 17, 'Read5': 2, 'Read6':  0},
'Read6': {'Read1':  0, 'Read3': 43, 'Read2':  0, 'Read5': 0, 'Read4':  1}}
```

…and the next read, …and the next read

Reconstruct the genome

```
Read1:      AGGTCGTAG
Read2:          CGTAGAGCTGGGAG
Read3:                  GGGAGGTTGAAA
```



```
genome:     AGGTCGTAGAGCTGGGAGGTTGAAA
```

`Seq` class

```
from Bio.Seq import Seq

dna = Seq("ATGGCCATTGTAATGGGCCGCTGA")
print(len(dna))
print("GCC" in dna)
print(dna[:3])
print(dna[-3:])
print(dna[:3] == "ATG")
print(type(dna[:3]))
```

`Seq` class

```
dna = Seq("ATGTGATAA")
print(dna.complement())
print(dna.reverse_complement())
print(dna.transcribe())
print(dna.translate())
print(dna.translate(to_stop=True))
```

`Seq` class

```
print(dna[:3])
print(dna[-3:])
print(dna[:3] == "ATG")
print(type(dna[:3]))
```

SeqRecord class

```
from Bio.SeqRecord import SeqRecord

record = SeqRecord(dna, id="N42", name="orf1",
                   description="A made-up ORF")
print(record)
print(record.id)
print(record.description)
print(record.seq)
```

`SeqFeature` class

```
# Find and extract the single CDS feature
cds_features = []
for feature in record.features:
    if feature.type == "CDS":
        cds_features.append(feature)

cds = cds_features[0]
```

`SeqFeature` class

```
cds_seq =
record.seq[cds.location.start:cds.location.end]

cds_seq = cds.extract(record.seq)
```

`SeqIO` module

```
from Bio import SeqIO

my_records = [
     SeqRecord(Seq("ATGGCTGA"),
               id="orf1", description="first"),
     SeqRecord(Seq("ATGAAATAA"),
               id="orf2", description="second"),
     SeqRecord(Seq("ATGTGATAA"),
               id="orf3", description="third"),
]

SeqIO.write(my_records, "my_orfs.fasta", "fasta")
```

`SeqIO` module

```
# Iterate over records
for rec in SeqIO.parse("my_orfs.fasta", "fasta"):
    print(rec.id, len(rec),

# Make list of records
list(SeqIO.parse("my_orfs.fasta", "fasta"))

# Get single record
rec = SeqIO.read("my_orfs.fasta", "fasta")
```

`SeqIO` module

```
record = SeqIO.read("NM_148900.4.gb","genbank")

SeqIO.write(record, "NM_148900.4.fasta", "fasta")
```

Entrez module

```
from Bio import Entrez

Entrez.email = "your@post.au.dk"    # use your own
address

handle = Entrez.efetch(db="nucleotide",  # database
search
                       id="NM_148900.4", # id of
sequence to download
                       rettype="gb",     # format of
downloaded record
                       retmode="text")

# read from the handle as you would from a file
seq_record = SeqIO.read(handle, "genbank")
```

Entrez module

```
from Bio import Entrez

# use your own address
Entrez.email = "your@post.au.dk"
handle = Entrez.efetch(db="nucleotide",
                       id="NM_148900.4",
                       rettype="gb",
                       retmode="text")

seq_record = SeqIO.read(handle, "genbank")

SeqIO.write(seq_record,
            "NM_148900.4.gb", "genbank")
```

Entrez module

```
handle = Entrez.esearch(
    db="nucleotide",
    term="FOXP2[Gene] AND Homo sapiens[Organism] AND
srcdb_refseq[prop]",
    retmax=10)

result = Entrez.read(handle)

for seq_id in result["IdList"]:
    handle = Entrez.efetch(db="nucleotide", id=seq_i
rettype="gb", retmode="text")
    record = SeqIO.read(handle, "genbank")
    print(record.id, record.description)
```